Data Manipulation with Pandas & Data Visualisation

Study guide · Exploration · Missing values · Matplotlib · Seaborn · Git & GitHub

Session 12 built a trustworthy table. This session is about **looking** at data properly: a repeatable first-look routine, deciding what to do with missing values, drawing and customising charts with **Matplotlib**, statistical charts with **Seaborn**, and exploring relationships with pair plots and heat maps. It ends with **Git and GitHub**, the tools that keep track of every version of your analysis code.

Most examples use two small public datasets that ship with Seaborn: `tips` (244 restaurant bills, a simple business dataset) and `iris` (150 flower measurements). Every example was run and every chart and terminal output shown is real.

Which chart answers which question?

Question you are asking	Chart	Code	Part
How does a value change over time or sequence?	line	`plt.plot(x, y)`	3
How do two numeric values relate?	scatter	`plt.scatter(x, y)`, `sns.scatterplot`	3, 5
How do a few categories compare?	bar	`plt.bar(categories, values)`	3
What is the shape of one numeric column?	histogram (+ KDE curve)	`sns.histplot(..., kde=True)`	4
How does a numeric column differ across groups? Any outliers?	box plot	`sns.boxplot(x=group, y=value)`	4
…and what does the whole distribution look like per group?	violin plot	`sns.violinplot(x=group, y=value)`	4
How do many numeric columns relate, pair by pair?	pair plot	`sns.pairplot(df, hue=...)`	5
Which numeric columns move together, and how strongly?	heat map of correlations	`sns.heatmap(df.corr(), annot=True)`	5

Part	Topic
1	Data exploration: a first-look routine
2	Handling missing values
3	Basic plots with Matplotlib: line, scatter, bar; customising and saving
4	Statistical plots with Seaborn: histogram, box plot, violin plot
5	Exploring relationships: pair plot and heat map
6	Practical exercise
7	Git and GitHub

Lesson plan (3 hours)

Time	Block	You do (trainer)	Learners do	Slides
0–25	Part 1 · Exploration	Run the first-look routine live on `tips`, one command at a time.	Predict each output; Try it 1.	3–5
25–45	Part 2 · Missing values	Delete vs impute vs fill forward; the pandas 3 `inplace` trap.	Try it 2.	6
45–80	Part 3 · Matplotlib	Line, scatter, bar; then customise one chart step by step; save it.	Try it 3.	7–12
80–90	Break			
90–115	Part 4 · Seaborn	Histogram with KDE, box plot, violin plot: same data, three views.	Try it 4.	13–17
115–135	Part 5 · Relationships	Pair plot on `iris`; correlation matrix and heat map on `tips`.	Try it 5.	18–21
135–155	Part 6 · Practical	Coach; learners build a mini EDA of `tips`.	Practical exercise.	22
155–180	Part 7 · Git & GitHub	Live: init, add, commit, log, branch, merge; then push to GitHub.	Try it 7.	23–47

Before class

☐ Libraries installed (from Session 12): `pip install pandas matplotlib seaborn`.

☐ Test `sns.load_dataset("tips")` on the classroom network. It downloads from GitHub. If it is blocked, use the `tips.csv` and `iris.csv` copies in the class bundle: `pd.read_csv("tips.csv")`.

☐ Git installed from git-scm.com (Windows learners get **Git Bash**). Check with `git --version`.

☐ Each learner has a free GitHub account for the last part of the session.

Part 1 Data exploration: a first-look routine

Before cleaning, modelling or charting, explore. Exploration helps you understand the structure and variables, find quality problems (missing values, outliers, wrong types), and spot patterns worth a closer look. The same routine works on any table.

> **Business case:** A restaurant has recorded 244 bills: the total bill, the tip, the party size, and who paid, on which day, at lunch or dinner. The manager wants to understand tipping before changing staff rosters.

p1_load.py

```
import pandas as pd
import seaborn as sns

tips = sns.load_dataset("tips")      # or: pd.read_csv("tips.csv")
print(tips.shape)
print(tips.head())
```

OUTPUT

```
(244, 7)
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
```

p1_types.py

```
print(tips.dtypes)
print(tips.isnull().sum())
```

OUTPUT

```
total_bill     float64
tip            float64
sex           category
smoker        category
day           category
time          category
size             int64
dtype: object
total_bill    0
tip           0
sex           0
smoker        0
day           0
time          0
size          0
dtype: int64
```

Seaborn stores the text columns as `category`, a pandas type for columns with a few repeated labels. No values are missing.

p1_describe.py

```
print(tips.describe().round(2))
```

OUTPUT

```
       total_bill     tip    size
count      244.00  244.00  244.00
mean        19.79    3.00    2.57
std          8.90    1.38    0.95
min          3.07    1.00    1.00
25%         13.35    2.00    2.00
50%         17.80    2.90    2.00
75%         24.13    3.56    3.00
max         50.81   10.00    6.00
```

p1_counts.py

```
print(tips["day"].value_counts())
print(tips["time"].value_counts())
```

OUTPUT

```
day
Sat     87
Sun     76
Thur    62
Fri     19
Name: count, dtype: int64
time
Dinner    176
Lunch      68
Name: count, dtype: int64
```

`value_counts()` is the categorical counterpart of `describe()`: how many rows fall in each label. Saturday and Sunday dominate, and most bills are at dinner.

A first look at shape, outliers and relationships

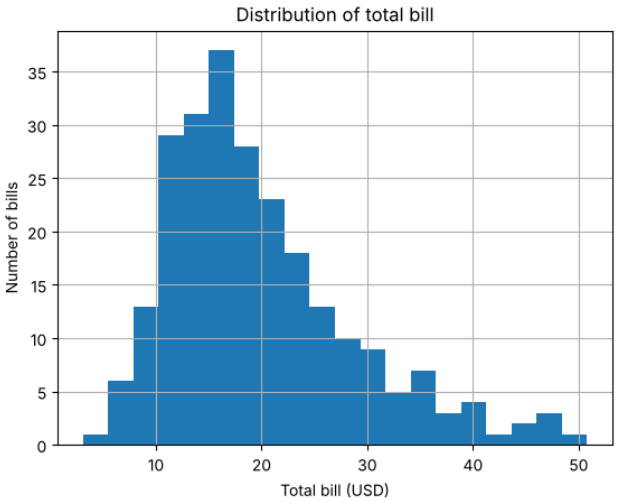
p1_hist.py

```
import matplotlib.pyplot as plt

tips["total_bill"].hist(bins=20)
plt.title("Distribution of total bill")
plt.xlabel("Total bill (USD)")
plt.ylabel("Number of bills")
plt.savefig("p1_hist.png", dpi=100)
plt.show()
```

OUTPUT

```
(nothing is printed)
```



Most bills are between about USD 10 and 25, with a tail of larger bills.

p1_corr.py

```
print(tips.corr(numeric_only=True).round(2))
```

OUTPUT

```
            total_bill   tip  size
total_bill        1.00  0.68  0.60
tip               0.68  1.00  0.49
size              0.60  0.49  1.00
```

`numeric_only=True` limits the correlation to number columns. (The deck's `df.corr()` raises an error on newer pandas when text columns are present.) Bill and tip move together fairly strongly (0.68). Part 5 explains correlation properly.

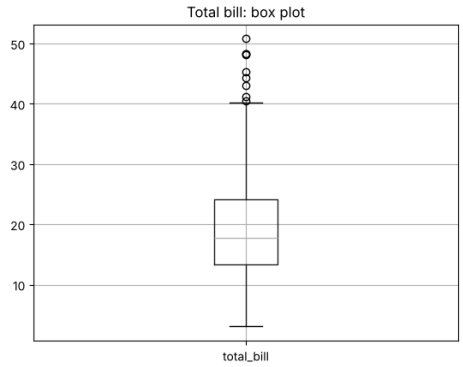
p1_box_scatter.py

```
tips.boxplot(column="total_bill")
plt.title("Total bill: box plot")
plt.savefig("p1_box.png", dpi=100)
plt.close()

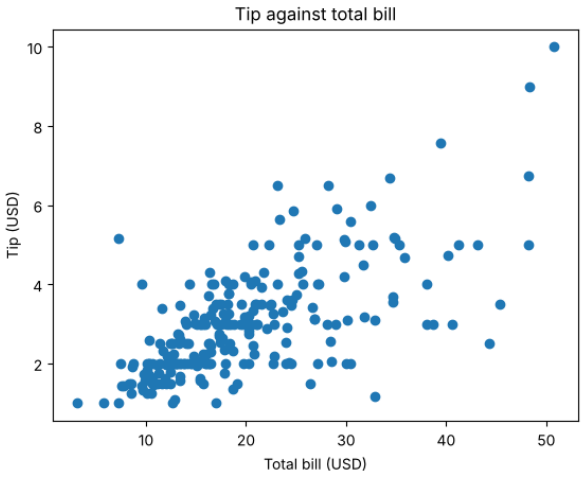
plt.scatter(tips["total_bill"], tips["tip"])
plt.xlabel("Total bill (USD)")
plt.ylabel("Tip (USD)")
plt.title("Tip against total bill")
plt.savefig("p1_scatter.png", dpi=100)
```

OUTPUT

```
(nothing is printed)
```



Dots above the top whisker are unusually large bills: worth a look, not automatically errors.



Bigger bills tend to get bigger tips, with plenty of variation.

Step	Code	Answers
Load	`pd.read_csv(...)` / `sns.load_dataset(...)`	can I read it?
Dimensions	`df.shape`	how many rows and columns?
Look	`df.head()`	what does a row look like?
Types	`df.dtypes`	is each column stored sensibly?
Missing	`df.isnull().sum()`	where are the gaps?
Summary	`df.describe()`	centre, spread, min and max of numbers
Categories	`df[col].value_counts()`	how many rows per label?
Distribution	`df[col].hist()`	what shape does a numeric column have?
Outliers	`df.boxplot(column=col)`	any unusual values?
Relationships	`df.corr(numeric_only=True)`, `plt.scatter`	which columns move together?

`isnull()` and `isna()` are the same function under two names.

> **TRY IT YOURSELF**
>
> Using `tips`: (a) What is the average bill on each day? (Hint: `groupby`.) (b) How many bills were from smokers and non-smokers? (c) What are the smallest and largest party sizes? (d) Create a new column `tip_pct` = tip ÷ total_bill × 100 and describe it.

Part 2 Handling missing values

Session 12's rule still holds: **missing is not zero**. Here are the main options and when each is reasonable.

> **Business case:** Two branches report daily unit sales and the shelf price. Some days are missing: a till was offline, a price was not captured, and the promo code is almost always blank.

p2_data.py

```
print(sales)
print(sales.isnull().sum())
```

OUTPUT

```
        date    branch  units  price promo_code
0 2026-10-01   Sandton   42.0   12.5        NaN
1 2026-10-02   Sandton    NaN   12.5        NaN
2 2026-10-03   Sandton   47.0    NaN      OCT10
3 2026-10-04   Sandton   51.0   12.5        NaN
4 2026-10-05  Rosebank    NaN   11.0        NaN
5 2026-10-06  Rosebank   38.0   11.0        NaN
6 2026-10-07  Rosebank   40.0    NaN        NaN
7 2026-10-08  Rosebank    NaN   11.0        NaN
date          0
branch        0
units         3
price         2
promo_code    7
dtype: int64
```

Option 1: deletion

p2_delete.py

```
print(sales.dropna().shape)
print(sales.dropna(subset=["units"]).shape)
print(sales.drop(columns="promo_code").columns.tolist())
```

OUTPUT

```
(0, 5)
(5, 5)
['date', 'branch', 'units', 'price']
```

`dropna()` with no arguments drops a row if **any** column is missing. Here that removes 7 of 8 rows because of the almost-empty `promo_code`. Name the columns that matter with `subset=`. A column that is mostly empty can be dropped instead, with `drop(columns=...)` (the deck's `axis=1` form does the same).

Option 2: imputation (fill with a value)

p2_mean.py

```
filled = sales.copy()
filled["units"] = filled["units"].fillna(filled["units"].mean())
print(filled[["date", "branch", "units"]])
```

OUTPUT

```
        date    branch  units
0 2026-10-01   Sandton   42.0
1 2026-10-02   Sandton   43.6
2 2026-10-03   Sandton   47.0
3 2026-10-04   Sandton   51.0
4 2026-10-05  Rosebank   43.6
5 2026-10-06  Rosebank   38.0
6 2026-10-07  Rosebank   40.0
7 2026-10-08  Rosebank   43.6
```

> **Watch out:** The deck writes `df["col"].fillna(value, inplace=True)`. In **pandas 3 this does nothing**: it fills a temporary copy and the DataFrame keeps its NaN (pandas prints a `ChainedAssignmentError` warning). Always assign the result back: `df["col"] = df["col"].fillna(value)`.

p2_inplace_trap.py

```
sales["units"].fillna(sales["units"].mean(), inplace=True)
print(sales["units"].isnull().sum(), "still missing")
```

OUTPUT

```
3 still missing
```

Mean imputation keeps the row, but it invents a value and ignores context: Rosebank sells fewer units than Sandton, yet both gaps get the same overall mean. A group mean is often more sensible:

p2_group_mean.py

```
by_branch = sales.copy()
by_branch["units"] = by_branch["units"].fillna(
    by_branch.groupby("branch")["units"].transform("mean"))
print(by_branch[["branch", "units"]])
```

OUTPUT

```
     branch      units
0   Sandton  42.000000
1   Sandton  46.666667
2   Sandton  47.000000
3   Sandton  51.000000
4  Rosebank  39.000000
5  Rosebank  38.000000
6  Rosebank  40.000000
7  Rosebank  39.000000
```

`transform("mean")` returns each row's own group mean, so every gap is filled with its branch's average.

Option 3: forward and backward fill

p2_ffill.py

```
prices = sales[["date", "branch", "price"]].copy()
prices["price_ffill"] = prices.groupby("branch")["price"].ffill()
prices["price_bfill"] = prices.groupby("branch")["price"].bfill()
print(prices)
```

OUTPUT

```
        date    branch  price  price_ffill  price_bfill
0 2026-10-01   Sandton   12.5         12.5         12.5
1 2026-10-02   Sandton   12.5         12.5         12.5
2 2026-10-03   Sandton    NaN         12.5         12.5
3 2026-10-04   Sandton   12.5         12.5         12.5
4 2026-10-05  Rosebank   11.0         11.0         11.0
5 2026-10-06  Rosebank   11.0         11.0         11.0
6 2026-10-07  Rosebank    NaN         11.0         11.0
7 2026-10-08  Rosebank   11.0         11.0         11.0
```

Forward fill copies the **last known** value down; backward fill copies the **next** one up. It suits values that persist until they change, such as a shelf price, an exchange rate or a status. Fill **within each group** so Sandton's price never leaks into Rosebank's rows. It does not suit values that vary day to day, such as units sold.

Option 4: an indicator variable

p2_indicator.py

```
flagged = sales.copy()
flagged["units_missing"] = flagged["units"].isnull().astype(int)
flagged["units"] = flagged["units"].fillna(flagged.groupby("branch")
["units"].transform("mean"))
print(flagged[["date", "branch", "units", "units_missing"]])
```

OUTPUT

```
        date    branch      units  units_missing
0 2026-10-01   Sandton  42.000000              0
1 2026-10-02   Sandton  46.666667              1
2 2026-10-03   Sandton  47.000000              0
3 2026-10-04   Sandton  51.000000              0
4 2026-10-05  Rosebank  39.000000              1
5 2026-10-06  Rosebank  38.000000              0
6 2026-10-07  Rosebank  40.000000              0
7 2026-10-08  Rosebank  39.000000              1
```

Record **that** a value was missing before filling it. Analysts can then exclude or check filled rows, and a later model can learn whether missingness itself carries information.

Strategy	Code	Use when	Risk
Drop rows	`df.dropna(subset=[...])`	few rows affected; the value is essential (S12's missing price)	losing data; bias if gaps are not random
Drop column	`df.drop(columns=col)`	the column is mostly empty or not needed	losing a field someone needs
Mean / median	`df[c] = df[c].fillna(df[c].median())`	a rough estimate is acceptable; few gaps	invents values; shrinks the spread
Group mean	`fillna(df.groupby(g)[c].transform("mean"))`	groups differ (branches, regions)	still an estimate
Forward / back fill	`df.groupby(g)[c].ffill()`	values that persist (prices, rates, status)	wrong for day-to-day measurements
Label	`df[c] = df[c].fillna("Not recorded")`	optional text fields	never for numbers or keys
Indicator	`df[c + "_missing"] = df[c].isnull().astype(int)`	alongside any fill, so it stays visible	an extra column to explain

TRY IT YOURSELF

Using `sales`: (a) Drop the `promo_code` column. (b) Fill the missing `price` values by forward fill within each branch. (c) Add `units_missing`, then fill missing units with the **median** of each branch. (d) Which rows would you show the manager as "estimated", and why?

Part 3 Basic plots with Matplotlib

Matplotlib is Python's core plotting library: flexible, customisable and publication quality. Seaborn and pandas' `.plot()` are built on it. Import it as `import matplotlib.pyplot as plt`.

> **Business case:** A shop's monthly sales (R thousands), January to June: 42, 38, 51, 47, 55, 60.

Line plot: change over time

p3_line.py

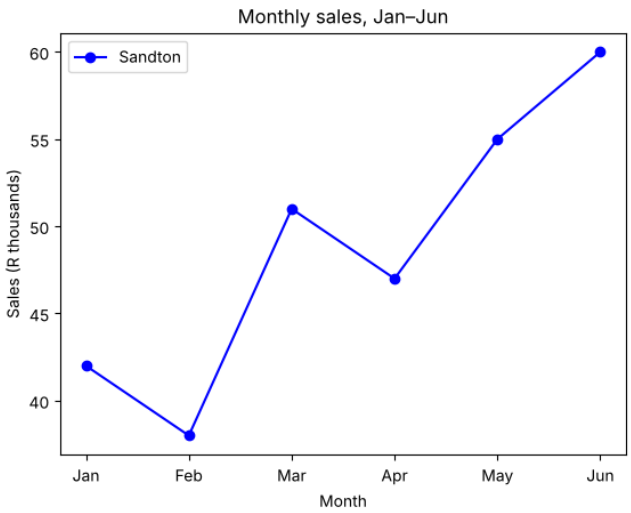
```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [42, 38, 51, 47, 55, 60]

plt.plot(months, sales, marker="o", linestyle="-", color="b", label="Sandton")
plt.xlabel("Month")
plt.ylabel("Sales (R thousands)")
plt.title("Monthly sales, Jan–Jun")
plt.legend()
plt.savefig("p3_line.png", dpi=100)
plt.show()
```

OUTPUT

```
(nothing is printed)
```



Scatter plot: two numeric measures

p3_scatter.py

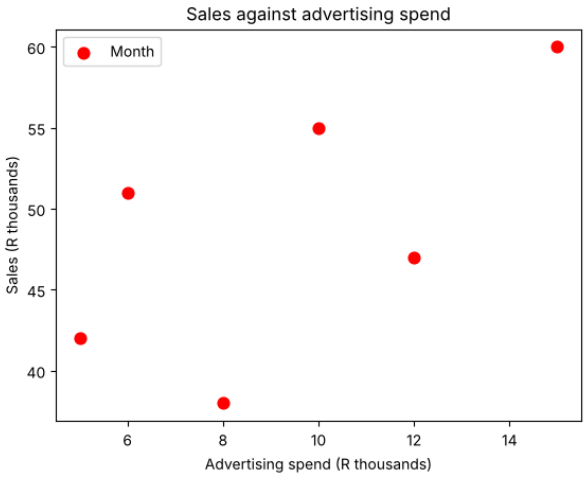
```
import matplotlib.pyplot as plt

ad_spend = [5, 8, 6, 12, 10, 15]          # R thousands
sales = [42, 38, 51, 47, 55, 60]

plt.scatter(ad_spend, sales, marker="o", color="r", s=60, label="Month")
plt.xlabel("Advertising spend (R thousands)")
plt.ylabel("Sales (R thousands)")
plt.title("Sales against advertising spend")
plt.legend()
plt.savefig("p3_scatter.png", dpi=100)
```

OUTPUT

```
(nothing is printed)
```



p3_scatter_error.py

```
import matplotlib.pyplot as plt
plt.scatter([5, 8, 6], [42, 38, 51], marker="o", markersize=8)
```

ERROR

```
Traceback (most recent call last):
  File "C:\Users\student\s13_eda\p3_scatter_error.py", line 2, in <module>
    plt.scatter([5, 8, 6], [42, 38, 51], marker="o", markersize=8)
    ~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: PathCollection.set() got an unexpected keyword argument 'markersize'
```

For `plt.scatter`, the marker size argument is `s=` (an area, e.g. `s=60`). `markersize=` belongs to `plt.plot`. The deck's customising slide mixes the two.

Bar chart: compare categories

p3_bar.py

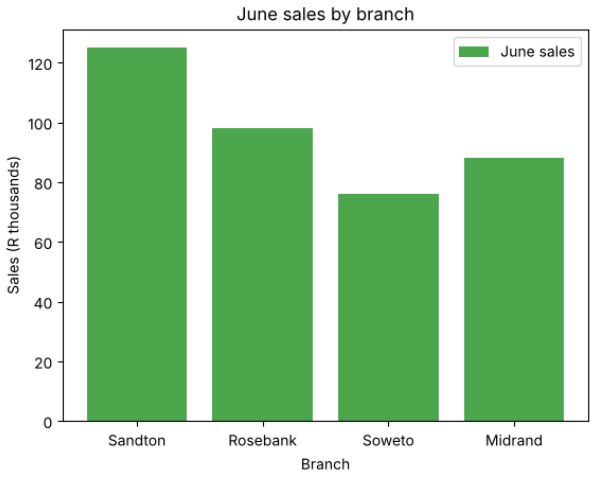
```
import matplotlib.pyplot as plt

branches = ["Sandton", "Rosebank", "Soweto", "Midrand"]
sales = [125, 98, 76, 88]

plt.bar(branches, sales, color="g", alpha=0.7, label="June sales")
plt.xlabel("Branch")
plt.ylabel("Sales (R thousands)")
plt.title("June sales by branch")
plt.legend()
plt.savefig("p3_bar.png", dpi=100)
```

OUTPUT

```
(nothing is printed)
```



Customising a plot

p3_custom.py

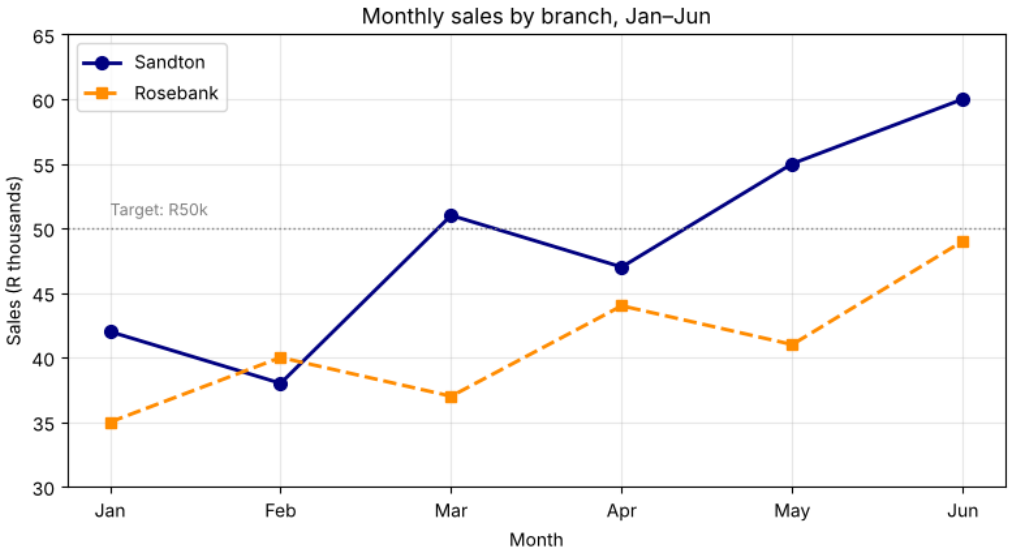
```python
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sandton = [42, 38, 51, 47, 55, 60]
rosebank = [35, 40, 37, 44, 41, 49]

plt.figure(figsize=(8, 4.5))
plt.plot(months, sandton, color="navy", linewidth=2, marker="o", markersize=7,
label="Sandton")
plt.plot(months, rosebank, color="darkorange", linewidth=2, linestyle="--", marker="s",
label="Rosebank")
plt.axhline(50, color="grey", linestyle=":", linewidth=1)
plt.text(0, 51, "Target: R50k", fontsize=9, color="grey")
plt.ylim(30, 65)
plt.grid(True, alpha=0.3)
plt.xlabel("Month")
plt.ylabel("Sales (R thousands)")
plt.title("Monthly sales by branch, Jan–Jun")
plt.legend()
plt.tight_layout()
plt.savefig("p3_custom.png", dpi=100)
```

OUTPUT

```
(nothing is printed)
```



Customisation	Code	Effect
Figure size	`plt.figure(figsize=(8, 4.5))`	width × height in inches; call it first
Labels and title	`plt.xlabel()`, `plt.ylabel()`, `plt.title()`	say what is plotted, with units
Several series	call `plt.plot` twice, each with `label=`	then `plt.legend()` to show the labels
Line	`linewidth=2`, `linestyle="--"` (`-` `--` `-.` `:`)	thickness and style
Markers	`marker="o"` (`o` `s` `^`), `markersize=7` (plot) / `s=60` (scatter)	point shape and size
Colour	`color="navy"`, `alpha=0.7`	colour and transparency
Axis range	`plt.xlim(0, 10)`, `plt.ylim(30, 65)`	fix the visible range
Grid	`plt.grid(True, alpha=0.3)`	light reference lines
Annotation	`plt.text(x, y, "note")`, `plt.axhline(50)`	label a point; draw a reference line
Colour maps	`plt.imshow(data, cmap="viridis")`	colour-coded grids (S11's photo, heat maps)

> **Watch out:** Bar charts should start at zero, so the bar lengths are comparable. Line charts may zoom in (`ylim`) when change is the point; say so in the title or axis if it matters.

Saving plots

p3_save.py

```
import matplotlib.pyplot as plt

plt.bar(["Sandton", "Rosebank"], [125, 98])
plt.title("June sales by branch")
plt.savefig("june_sales.png", dpi=150, bbox_inches="tight")
plt.savefig("june_sales.pdf", bbox_inches="tight")
plt.show()

import os
print(os.path.exists("june_sales.png"), os.path.exists("june_sales.pdf"))
```

OUTPUT

```
True True
```

The file extension chooses the format: `.png` for slides and documents, `.pdf` or `.svg` for print-quality vector graphics. `dpi` sets the resolution of PNGs; `bbox_inches="tight"` trims spare white space. Call `savefig` **before** `plt.show()`: in a script, closing the window clears the figure.

pyplot style or figure-and-axes style?

The deck uses the quick **pyplot** style (`plt.plot`, `plt.title`). Session 11 used `fig, ax = plt.subplots()` and `ax.plot`, `ax.set_title`. Both draw the same charts. Use `fig, ax` when you need several charts in one figure or want to be explicit about which chart you are changing.

TRY IT YOURSELF

Plot two lines on one chart: Sandton `[42, 38, 51, 47, 55, 60]` and Soweto `[30, 33, 29, 36, 40, 38]` for January–June. Give each a label and a different line style, add a legend, a grid, a title and axis labels with units, set the y-axis from 0 to 70, and save the chart as both `branches.png` and `branches.pdf`.

Part 4 Statistical plots with Seaborn

Seaborn is built on Matplotlib and designed for statistical charts. It works directly with DataFrame columns, summarises data for you, and has attractive defaults. You still finish charts with Matplotlib commands (`plt.title`, `plt.xlabel`).

	Matplotlib	Seaborn
Input	lists or arrays you prepare	a DataFrame and column names: `data=df, x="col"`
Statistics	you compute them first	built in (counts, KDE curves, quartiles, means)
Groups and colour	a loop or several calls	one argument: `hue="group"`
Control	every detail	good defaults; fine-tune with Matplotlib
Use it for	custom or simple charts	fast exploration of distributions and relationships

Histogram with a density curve

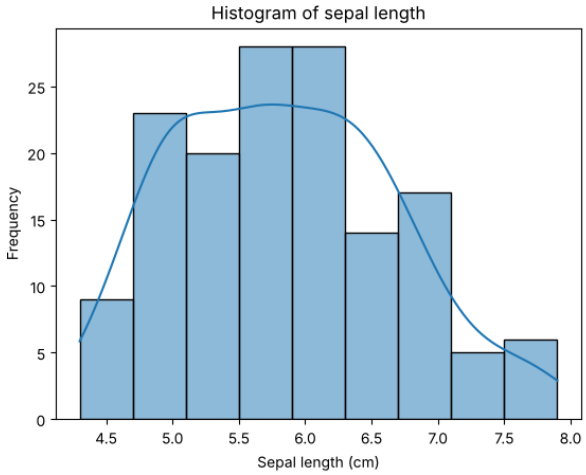
p4_hist.py

```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset("iris")
sns.histplot(data=data, x="sepal_length", kde=True)
plt.xlabel("Sepal length (cm)")
plt.ylabel("Frequency")
plt.title("Histogram of sepal length")
plt.savefig("p4_hist.png", dpi=100)
plt.show()
```

OUTPUT

```
(nothing is printed)
```



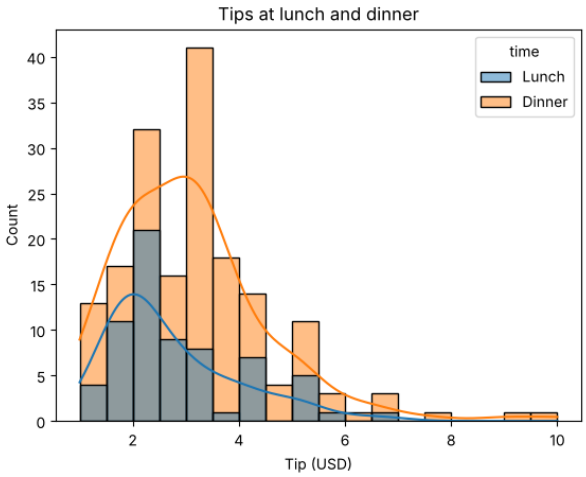
`kde=True` adds a smoothed curve, an estimate of the distribution's shape.

p4_hist_hue.py

```
sns.histplot(data=tips, x="tip", hue="time", kde=True)
plt.xlabel("Tip (USD)")
plt.title("Tips at lunch and dinner")
plt.savefig("p4_hist_hue.png", dpi=100)
```

OUTPUT

```
(nothing is printed)
```



`hue=` splits one histogram by a category: a business question in one argument.

Box plot: compare groups

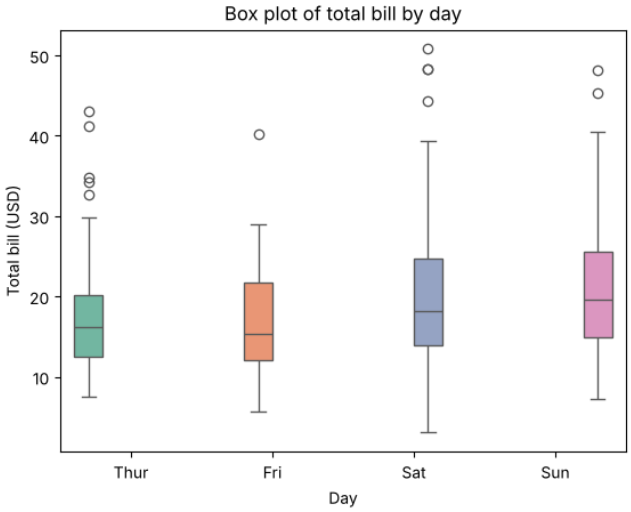
p4_box.py

```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset("tips")
sns.boxplot(data=data, x="day", y="total_bill", hue="day", palette="Set2", legend=False)
plt.xlabel("Day")
plt.ylabel("Total bill (USD)")
plt.title("Box plot of total bill by day")
plt.savefig("p4_box.png", dpi=100)
```

OUTPUT

```
(nothing is printed)
```



Box: middle half of bills. Line: median. Whiskers: typical range. Dots: unusually large bills.

> **Watch out:** The deck passes `palette=` without `hue=`. Seaborn 0.13 warns that this will stop working. Add `hue=` with the same column as `x` and `legend=False`, as above.

Violin plot: the whole distribution per group

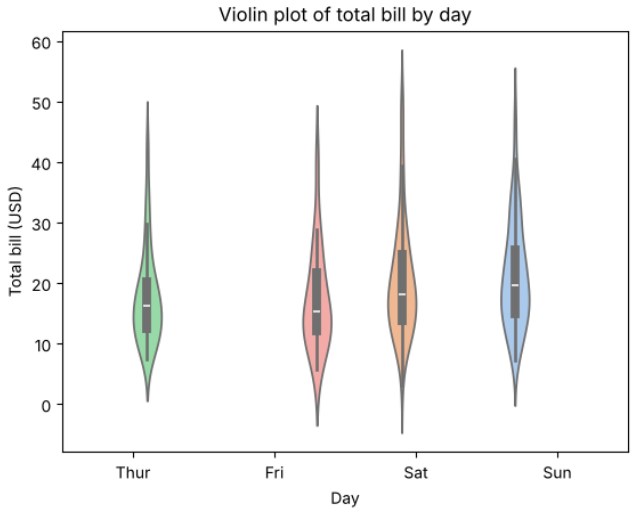
p4_violin.py

```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset("tips")
sns.violinplot(data=data, x="day", y="total_bill", hue="day", palette="pastel",
legend=False)
plt.xlabel("Day")
plt.ylabel("Total bill (USD)")
plt.title("Violin plot of total bill by day")
plt.savefig("p4_violin.png", dpi=100)
```

OUTPUT

```
(nothing is printed)
```



Width shows how many bills sit at each value; the small box inside is a box plot.

Chart	Shows	Choose it when
Histogram	counts in ranges for **one** numeric column	you want the overall shape, skew and gaps
Box plot	median, quartiles and outliers per group	comparing many groups quickly; spotting outliers
Violin plot	the full distribution shape per group	groups may have two peaks or odd shapes a box hides

TRY IT YOURSELF

(a) Draw a histogram of `tip` with a KDE curve and proper labels. (b) Draw a violin plot of `tip` by `time` (Lunch, Dinner). (c) Draw a box plot of `total_bill` by `day`, split by `smoker` using `hue="smoker"`. What do you notice?

Part 5 Exploring relationships: pair plot and heat map

Pair plot: every numeric pair at once

A **pair plot** draws a scatter plot for every pair of numeric columns, with each column's distribution on the diagonal. `hue=` colours the points by group.

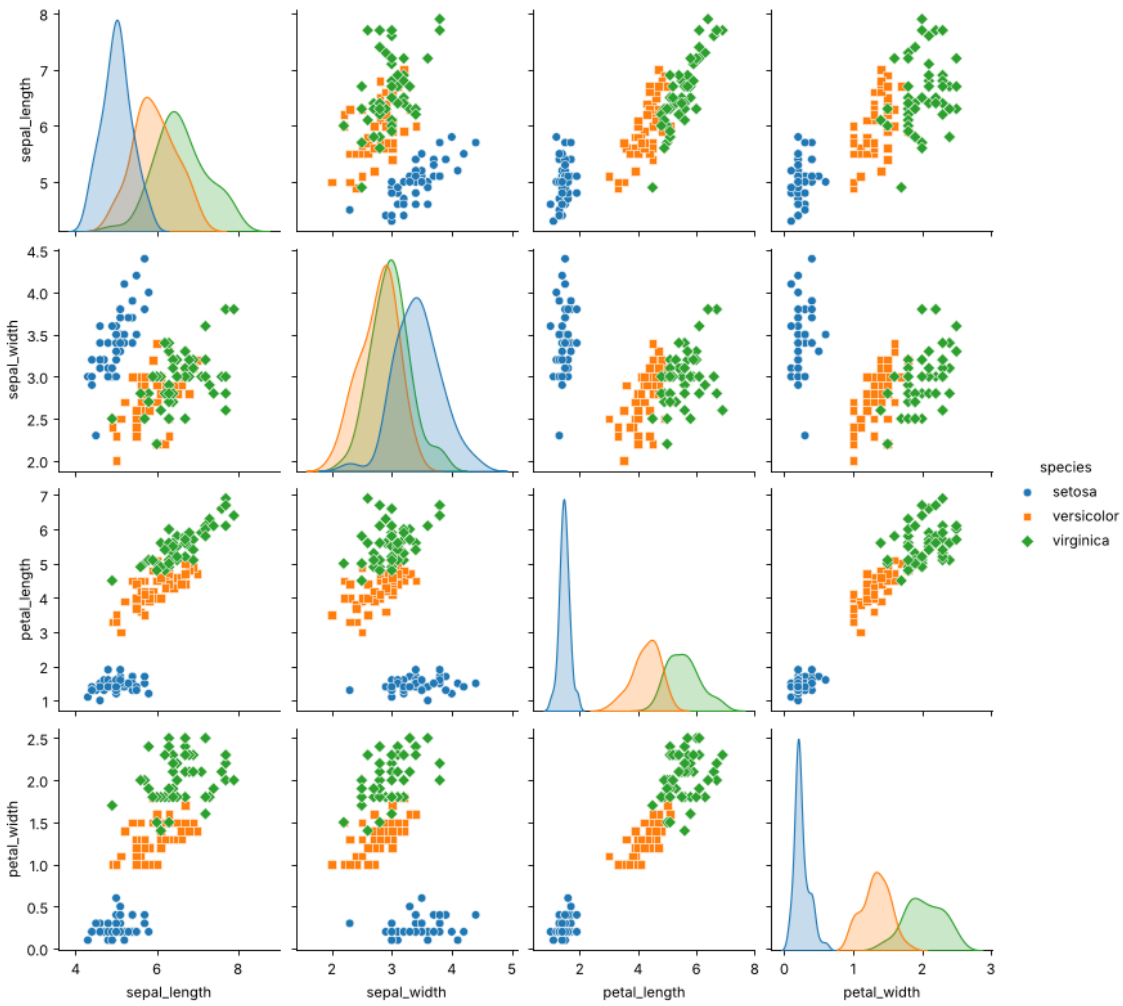
p5_pairplot.py

```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset("iris")
sns.pairplot(data, hue="species", markers=["o", "s", "D"])
plt.savefig("p5_pairplot.png", dpi=80)
plt.show()
```

OUTPUT

```
(nothing is printed)
```



Petal length and width separate the three species almost perfectly; sepal width overlaps much more.

Read a pair plot row by row: each cell shows the row's column (y) against the column's column (x). It is an exploration tool, ideal for finding which variables separate groups or move together before any modelling.

Correlation and the heat map

Correlation measures how strongly two numeric columns move together in a straight line, from −1 to +1:

Value	Meaning	Example
close to +1	strong positive: when one rises, the other rises	bill and tip
close to 0	little straight-line relationship	
close to −1	strong negative: when one rises, the other falls	price and quantity demanded

p5_heatmap.py

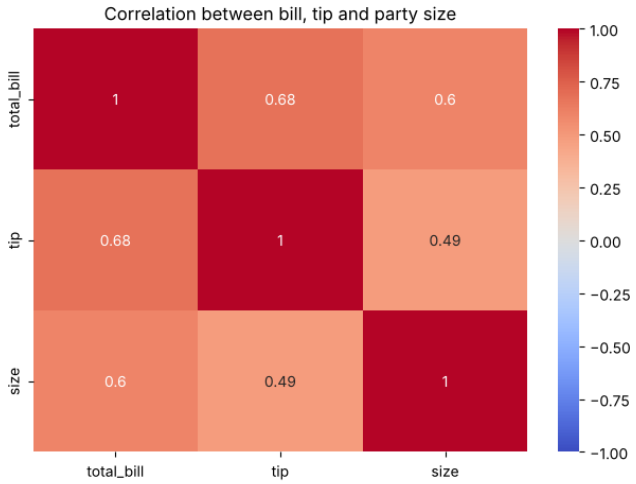
```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset("tips")
numeric_data = data[["total_bill", "tip", "size"]]
correlation_matrix = numeric_data.corr()
print(correlation_matrix.round(2))

sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Correlation between bill, tip and party size")
plt.tight_layout()
plt.savefig("p5_heatmap.png", dpi=100)
```

OUTPUT

```
            total_bill   tip  size
total_bill        1.00  0.68  0.60
tip               0.68  1.00  0.49
size              0.60  0.49  1.00
```



Warm = positive, cool = negative. `vmin=-1, vmax=1` fixes the colour scale so the colours mean the same in every heat map.

> **Watch out: Correlation is not causation.** Bigger parties order more food *and* leave bigger tips; the tip does not cause the bill. Correlation also only measures straight-line relationships, and one outlier can move it a lot. Always look at the scatter plot too.

In later sessions, heat maps help choose features for models and spot **multicollinearity**: two input columns so strongly correlated that they carry the same information.

TRY IT YOURSELF

(a) Draw a heat map of the correlations between the four numeric `iris` columns (`iris.drop(columns="species")`). Which pair is most strongly correlated? (b) Draw a pair plot of `tips` with `hue="time"`. Does lunch look different from dinner?

Part 6 Practical exercise: a mini EDA of the restaurant bills

> **Trainer note:** Learners work alone or in pairs on a fresh notebook or script. Ask for one sentence of interpretation under every chart: the pattern, the population and one limit.

The restaurant manager asks: *When do we earn the most, and who tips best?* Answer with the `tips` data:

1. Run the first-look routine (Part 1) and write down two observations.
2. Add `tip_pct` = tip ÷ total_bill × 100.
3. Which **day** has the highest total takings (sum of bills)? Show it as a bar chart, with bars starting at zero.
4. Compare `tip_pct` for smokers and non-smokers with a box plot. Is there a clear difference?
5. Does party `size` relate to `tip_pct`? Use a scatter plot or a correlation.
6. Combine three charts into one figure with `fig, axes = plt.subplots(1, 3, figsize=(15, 4))`, and save it as `restaurant_eda.png`.
7. Write three findings and one limitation.

Answers and a reference figure are in the appendix.

Part 7 Git and GitHub

Git is a version-control system: it records every saved version of your project, who changed what, and lets you go back to any earlier version. **GitHub** is a website that hosts Git repositories online for backup, sharing and collaboration. Git was created by Linus Torvalds in 2005 to manage the Linux kernel.

Term	Meaning
repository (repo)	a project folder whose history Git tracks (in a hidden `.git` folder)
working folder	the files as you see and edit them
staging area	the changes you have chosen for the next snapshot (`git add`)
commit	a permanent snapshot of the staged changes, with a message
branch	a separate line of work, e.g. a new feature, without disturbing `main`
merge	bring a branch's commits into another branch
remote / push / pull	a copy on GitHub; send your commits up / bring others' commits down

Edit files → **git add** → **git commit** → **git push**

Install Git from git-scm.com. On Windows this also installs **Git Bash**, a terminal for the commands below; in VS Code choose it from the terminal drop-down. The outputs below are real, from a fresh repository.

One-time setup

TERMINAL (GIT BASH)

```
$ git --version
git version 2.43.0
$ git config --global user.name "Thandi Nkosi"
$ git config --global user.email "thandi@example.com"
$ git config --global init.defaultBranch main
$ git config --global --list
user.name=Thandi Nkosi
user.email=thandi@example.com
init.defaultbranch=main
```

Your name and e-mail are stamped on every commit. Older Git versions call the first branch `master`; GitHub and most teams now use `main`.

Create a repository and make the first commit

TERMINAL (GIT BASH)

```
$ mkdir myproject
$ cd myproject
$ git init
Initialized empty Git repository in /c/Users/student/myproject/.git/
```

TERMINAL · CREATE TWO FILES, THEN CHECK

```
$ printf '# Restaurant tips EDA\n\nExploratory analysis of the tips dataset.\n' >
README.md ...
$ ls
README.md
eda.py
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        eda.py

nothing added to commit but untracked files present (use "git add" to track)
```

The first line creates `README.md` and `eda.py`; in class, simply create them in VS Code.

TERMINAL (GIT BASH)

```
$ git add README.md
$ git status --short
A  README.md
?? eda.py
$ git add eda.py
$ git commit -m "Add README and first EDA script"
[main (root-commit) a463ac7] Add README and first EDA script
 2 files changed, 7 insertions(+)
 create mode 100644 README.md
 create mode 100644 eda.py
```

`git add` moves changes to the staging area; `git commit` saves them as a snapshot. Write messages that say **what changed and why**, e.g. "Add box plot of bills by day", not "update".

Change a file, check, commit again

TERMINAL (GIT BASH)

```
$ printf 'print(tips.describe())\n' >> eda.py
$ git status --short
 M eda.py
$ git diff
diff --git a/eda.py b/eda.py
index ad02c60..13738c8 100644
--- a/eda.py
+++ b/eda.py
@@ -2,3 +2,4 @@ import seaborn as sns

 tips = sns.load_dataset("tips")
 print(tips.shape)
+print(tips.describe())
$ git add --all
$ git commit -m "Print summary statistics"
[main d0890ff] Print summary statistics
 1 file changed, 1 insertion(+)
$ git log --oneline
d0890ff Print summary statistics
a463ac7 Add README and first EDA script
```

Short status flag	Meaning
`??`	untracked: Git does not know the file yet
`A`	added to the staging area
`M`	modified since the last commit
`D`	deleted

`git diff` shows exactly which lines changed before you stage them. `git log` lists the history; `--oneline` gives one line per commit.

Branches: work on a feature safely

TERMINAL (GIT BASH)

```
$ git branch add-charts
$ git checkout add-charts
Switched to branch 'add-charts'
$ printf 'import matplotlib.pyplot as plt\n\nsns.boxplot(data=tips, x="day",
y="total_bill")\nplt.savefig("box_by_day.png")\n' > charts.py
$ git add charts.py
$ git commit -m "Add box plot of total bill by day"
[add-charts eb01a13] Add box plot of total bill by day
 1 file changed, 4 insertions(+)
 create mode 100644 charts.py
$ git branch
* add-charts
  main
```

TERMINAL · BACK TO MAIN, THEN MERGE

```
$ git checkout main
Switched to branch 'main'
$ ls
README.md
eda.py
$ git merge add-charts
Updating d0890ff..eb01a13
Fast-forward
 charts.py | 4 ++++
 1 file changed, 4 insertions(+)
 create mode 100644 charts.py
$ ls
README.md
charts.py
eda.py
$ git log --oneline --graph
* eb01a13 Add box plot of total bill by day
* d0890ff Print summary statistics
* a463ac7 Add README and first EDA script
```

On `main`, `charts.py` does not exist yet: the commit lives only on the branch. `git merge add-charts` brings it in. `git switch` is a newer, clearer alternative to `git checkout` for changing branches.

A .gitignore for data science projects

TERMINAL (GIT BASH)

```
$ printf 'data/\n*.csv\n__pycache__/\n.ipynb_checkpoints/\n.venv/\n.env\n' > .gitignore
$ cat .gitignore
data/
*.csv
__pycache__/
.ipynb_checkpoints/
.venv/
.env
$ git add .gitignore
$ git commit -m "Ignore data, caches and secrets"
[main 73078cc] Ignore data, caches and secrets
 1 file changed, 6 insertions(+)
 create mode 100644 .gitignore
```

Watch out: Do not commit large data files, virtual environments or secrets (API keys, passwords). List them in `.gitignore` **before** the first commit. Once something is pushed to GitHub, deleting it later does not remove it from the history.

Help

TERMINAL · OPTIONS FOR ONE COMMAND

```
$ git commit -h 2>&1 | head -8
usage: git commit [-a | --interactive | --patch] [-s] [-v] [-u<mode>] [--amend]
                  [--dry-run] [(-c | -C | --squash) <commit> | --fixup [(amend|
reword):]<commit>)]
                  [-F <file> | -m <msg>] [--reset-author] [--allow-empty]
                  [--allow-empty-message] [--no-verify] [-e] [--author=<author>]
                  [--date=<date>] [--cleanup=<mode>] [--[no-]status]
                  [-i | -o] [--pathspec-from-file=<file> [--pathspec-file-nul]]
                  [(--trailer <token>[(=|:)<value>])...] [-S[<keyid>]]
                  [--] [<pathspec>...]
```

`git <command> -h` lists a command's options; `git help --all` lists every command.

GitHub: put the repository online

1. Sign in at github.com and click **New repository**.
2. Name it `myproject`. Leave *Add a README* unticked, because the local repository already has one.
3. Copy the repository address GitHub shows, then run the commands below in your project folder.
4. Refresh the GitHub page: your files and commit history are online.

```
git remote add origin https://github.com/<your-username>/myproject.git
git push -u origin main
```

`origin` is the conventional name for the GitHub copy. `-u` links your local `main` to it, so afterwards `git push` and `git pull` need no extra arguments. GitHub asks you to sign in the first time (a browser window or a personal access token, not your password).

Command	Does
`git clone <url>`	copy an existing GitHub repository to your computer
`git pull`	bring down and merge commits others have pushed
`git push`	send your new commits up
`git remote -v`	show which online repository this folder is linked to

Git command summary

Task	Command
Check the installed version	`git --version`
Set your identity	`git config --global user.name "..."` / `user.email`
Start tracking a folder	`git init`
See what changed	`git status`, `git status --short`, `git diff`
Stage changes	`git add <file>` or `git add --all`
Save a snapshot	`git commit -m "message"`
View history	`git log`, `git log --oneline`
Create / switch branch	`git branch <name>`, `git checkout <name>`
Combine branches	`git merge <branch>`
Link and upload to GitHub	`git remote add origin <url>`, `git push -u origin main`

TRY IT YOURSELF

In a new folder `tips_eda`: initialise a repository, add a `.gitignore` that ignores `*.csv`, create `explore.py` with your Part 1 code and commit it. Create a branch `heatmap`, add a heat map script, commit, switch back to `main` and merge. Finish with `git log --oneline` and, if you have a GitHub account, push it.

Close

Check your understanding

1. Name five commands in a first-look routine and what each one tells you.
2. When is forward fill appropriate, and when would it mislead?
3. Why does `df["col"].fillna(value, inplace=True)` not work in pandas 3, and what do you write instead?
4. Which chart would you use to compare the spread of bills across days, and why might you choose a violin over a box plot?
5. A heat map shows a correlation of 0.68 between bill and tip. What can and can't you conclude?
6. What is the difference between `git add` and `git commit`, and between `git commit` and `git push`?

Appendix: Answers to the exercises

Part 1

a1.py

```
print(tips.groupby("day", observed=True)["total_bill"].mean().round(2))
print(tips["smoker"].value_counts())
print(tips["size"].min(), tips["size"].max())
tips["tip_pct"] = tips["tip"] / tips["total_bill"] * 100
print(tips["tip_pct"].describe().round(2))
```

OUTPUT

```
day
Thur    17.68
Fri     17.15
Sat     20.44
Sun     21.41
Name: total_bill, dtype: float64
smoker
No     151
Yes     93
Name: count, dtype: int64
1 6
count    244.00
mean      16.08
std        6.11
min        3.56
25%       12.91
50%       15.48
75%       19.15
max       71.03
Name: tip_pct, dtype: float64
```

`observed=True` lists only the days that appear in the data (relevant for `category` columns).

Part 2

a2.py

```
answer = sales.drop(columns="promo_code")
answer["price"] = answer.groupby("branch")["price"].ffill()
answer["units_missing"] = answer["units"].isnull().astype(int)
answer["units"] = answer["units"].fillna(answer.groupby("branch")
["units"].transform("median"))
print(answer)
```

OUTPUT

```
        date    branch  units  price  units_missing
0 2026-10-01   Sandton   42.0   12.5              0
1 2026-10-02   Sandton   47.0   12.5              1
2 2026-10-03   Sandton   47.0   12.5              0
3 2026-10-04   Sandton   51.0   12.5              0
4 2026-10-05  Rosebank   39.0   11.0              1
5 2026-10-06  Rosebank   38.0   11.0              0
6 2026-10-07  Rosebank   40.0   11.0              0
7 2026-10-08  Rosebank   39.0   11.0              1
```

(d) Show the rows where `units_missing` is 1 as estimated: their units are a branch median, not a recorded sale.

Part 3

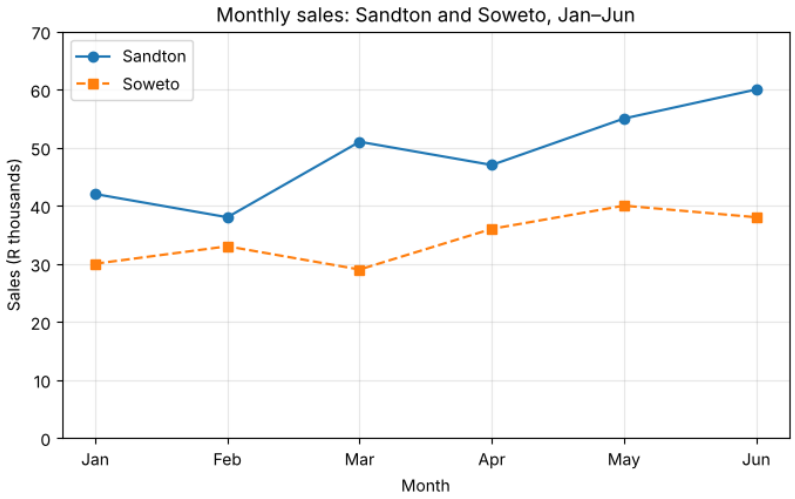
a3.py

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
plt.figure(figsize=(8, 4.5))
plt.plot(months, [42, 38, 51, 47, 55, 60], linestyle="-", marker="o", label="Sandton")
plt.plot(months, [30, 33, 29, 36, 40, 38], linestyle="--", marker="s", label="Soweto")
plt.ylim(0, 70)
plt.grid(True, alpha=0.3)
plt.xlabel("Month")
plt.ylabel("Sales (R thousands)")
plt.title("Monthly sales: Sandton and Soweto, Jan–Jun")
plt.legend()
plt.savefig("branches.png", dpi=100, bbox_inches="tight")
plt.savefig("branches.pdf", bbox_inches="tight")
```

OUTPUT

```
(nothing is printed)
```



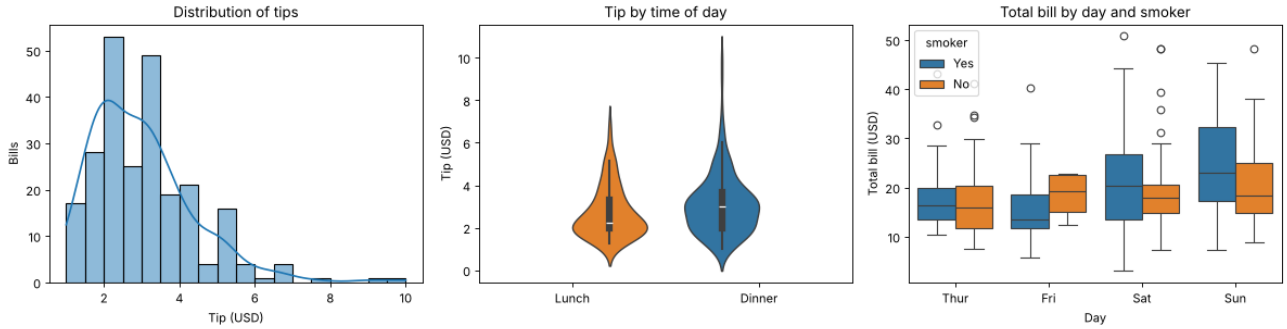
Part 4

a4.py

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
sns.histplot(data=tips, x="tip", kde=True, ax=axes[0])
axes[0].set(title="Distribution of tips", xlabel="Tip (USD)", ylabel="Bills")
sns.violinplot(data=tips, x="time", y="tip", hue="time", legend=False, ax=axes[1])
axes[1].set(title="Tip by time of day", xlabel="", ylabel="Tip (USD)")
sns.boxplot(data=tips, x="day", y="total_bill", hue="smoker", ax=axes[2])
axes[2].set(title="Total bill by day and smoker", xlabel="Day", ylabel="Total bill (USD)")
fig.tight_layout()
fig.savefig("a4.png", dpi=80)
```

OUTPUT

```
(nothing is printed)
```



With `ax=`, each Seaborn chart goes into its own panel of one figure.

Part 5

a5.py

```
corr = iris.drop(columns="species").corr()
print(corr.round(2))
stacked = corr.where(~(corr == 1)).stack()
print("Strongest pair:", stacked.idxmax(), round(stacked.max(), 2))
```

OUTPUT

```
              sepal_length  sepal_width  petal_length  petal_width
sepal_length          1.00        -0.12          0.87         0.82
sepal_width          -0.12         1.00         -0.43        -0.37
petal_length          0.87        -0.43          1.00         0.96
petal_width           0.82        -0.37          0.96         1.00
Strongest pair: ('petal_length', 'petal_width') 0.96
```

Petal length and petal width are almost perfectly correlated (0.96). In the `tips` pair plot, dinner bills reach higher values, but lunch and dinner follow a similar bill–tip pattern.

Part 6: practical exercise

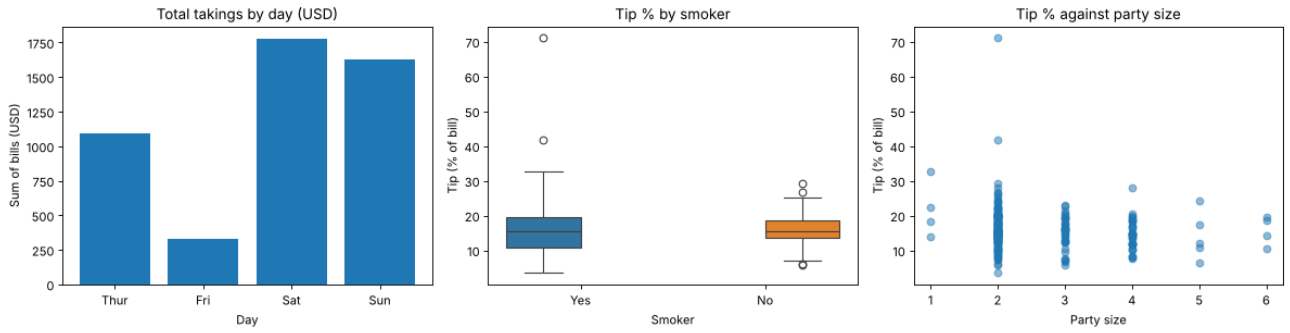
a6.py

```
tips["tip_pct"] = tips["tip"] / tips["total_bill"] * 100
takings = tips.groupby("day", observed=True)["total_bill"].sum().round(2)
print(takings)
print(tips.groupby("smoker", observed=True)["tip_pct"].median().round(2))
print(round(tips["size"].corr(tips["tip_pct"]), 2))

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].bar(takings.index.astype(str), takings.values)
axes[0].set(title="Total takings by day (USD)", xlabel="Day", ylabel="Sum of bills (USD)")
sns.boxplot(data=tips, x="smoker", y="tip_pct", hue="smoker", legend=False, ax=axes[1])
axes[1].set(title="Tip % by smoker", xlabel="Smoker", ylabel="Tip (% of bill)")
axes[2].scatter(tips["size"], tips["tip_pct"], alpha=0.5)
axes[2].set(title="Tip % against party size", xlabel="Party size", ylabel="Tip (% of
bill)")
fig.tight_layout()
fig.savefig("restaurant_eda.png", dpi=80)
```

OUTPUT

```
day
Thur    1096.33
Fri      325.88
Sat     1778.40
Sun     1627.16
Name: total_bill, dtype: float64
smoker
Yes    15.38
No     15.56
Name: tip_pct, dtype: float64
-0.14
```



Findings: Saturday brings in the most takings, then Sunday. Median tip percentages for smokers and non-smokers are similar, though smokers' tips vary more. Larger parties tip a slightly smaller percentage, but the relationship is weak (correlation −0.14). **Limitation:** 244 bills from one restaurant; the data show patterns, not why customers tip as they do.

Part 7

```
mkdir tips_eda
cd tips_eda
git init
echo "*.csv" > .gitignore
# create explore.py in VS Code
git add .gitignore explore.py
git commit -m "Add first exploration script"
git branch heatmap
git checkout heatmap
# create heatmap.py
git add heatmap.py
git commit -m "Add correlation heat map"
git checkout main
git merge heatmap
git log --oneline
git remote add origin https://github.com/<your-username>/tips_eda.git
git push -u origin main
```